articulation_points

```
// Articulation(n): undirected graph on nodes 0..n-1
// add(a,b): edge between a and b
// get(): all articulation points (nodes whose removal disconnects their component) in increasing order
struct Articulation{
  int t=0;vc<vi>g;vi tin,low,cut;
  Articulation(int n):g(n),tin(n),low(n),cut(n){}
  void add(int a,int b){
    g[a].pb(b);
    g[b].pb(a);
  }
  void dfs(int u,int p){
    tin[u]=low[u]=++t;
    int ch=0;
    for(int v:g[u])if(v!=p){
      if(tin[v])low[u]=min(low[u],tin[v]);
      else{
        dfs(v,u);
        low[u]=min(low[u],low[v]);
        if(low[v]>=tin[u]&&p>=0)cut[u]=1;
        ch++;
      }
    }
    if(p<0&&ch>1)cut[u]=1;
  }
  vi get(){
    rep(i,sz(g))if(!tin[i])dfs(i,-1);
    vi r;
    rep(i,sz(g))if(cut[i])r.pb(i);
    return r;
  }
};
```

convex_hull

```
// convex_hull(a,include_collinear): replaces a with its convex hull in counter-clockwise order, starting at the smallest point
// include_collinear: also keep points lying on hull edges
// cross(a,b,c): >0 if a->b->c turns left, <0 if right, 0 if collinear
struct pt{
  ll x,y;
  bool operator<(pt o)const{return pll(x,y)<pll(o.x,o.y);}
  bool operator==(pt o)const{return x==o.x&&y==o.y;}
};
ll cross(pt a,pt b,pt c){return(b.x-a.x)*(c.y-a.y)-(b.y-a.y)*(c.x-a.x);}

void convex_hull(vc<pt>&a,bool include_collinear=0){
  sort(all(a));a.erase(unique(all(a)),a.end());
  if(sz(a)<3)return;
  vc<pt>h;
  rep(it,2){
    int s=sz(h);
    for(pt p:a){
      while(sz(h)>=s+2&&cross(h[sz(h)-2],h.back(),p)<!include_collinear)h.pop_back();
      h.pb(p);
    }
    h.pop_back();reverse(all(a));
  }
  if(sz(h)>sz(a))h.resize(sz(a));
  a=h;
}
```

euclid

```
// gcd(a,b,x,y): returns gcd(a,b) and sets x,y so that a*x+b*y=gcd(a,b)
ll gcd(ll a,ll b,ll&x,ll&y){
  if(!b)return x=1,y=0,a;
  ll d=gcd(b,a%b,y,x);
  y-=a/b*x;
  return d;
}
```

fenwick

```
// FT(n): n zeros, upd(i,x): a[i]+=x, qry(i): a[0]+...+a[i-1]
struct FT{
  vl s;
  FT(int n):s(n){}
  void upd(int i,ll x){
    for(;i<sz(s);i|=i+1)s[i]+=x;
  }
  ll qry(int i){
    ll r=0;
    for(;i>0;i&=i-1)r+=s[i-1];
    return r;
  }
};

// FT2(h): 2D sums over points with 0<=x<h, all points that will be updated must be known in advance
// inform(x,y) for every such point, then init(), then use upd and qry
// upd(x,y,d): value at (x,y)+=d
// qry(x,y): sum over x'<x,y'<y, qry(x1,y1,x2,y2): sum over x1<=x'<x2,y1<=y'<y2
struct FT2{
  vc<vi>ys;vc<FT>ft;
  FT2(int h):ys(h){}
  void inform(int x,int y){
    for(;x<sz(ys);x|=x+1)ys[x].pb(y);
  }
  void init(){
    for(vi&v:ys)sort(all(v)),v.erase(unique(all(v)),v.end()),ft.pb(sz(v));
  }
  int ind(int x,int y){
    return lower_bound(all(ys[x]),y)-ys[x].begin();
  }
  void upd(int x,int y,ll dif){
    for(;x<sz(ys);x|=x+1)ft[x].upd(ind(x,y),dif);
  }
  ll qry(int x,int y){
    ll r=0;
    for(;x;x&=x-1)r+=ft[x-1].qry(ind(x-1,y));
    return r;
  }
  ll qry(int x1,int y1,int x2,int y2){
    return qry(x2,y2)-qry(x2,y1)-qry(x1,y2)+qry(x1,y1);
  }
};
```

fft

```
// multiply(a,b): coefficients of the product of polynomials a and b
// exact while (sum of a[i]^2 + sum of b[i]^2)*log2(n) < 9e14
typedef complex<double> C;
void fft(vc<C>&a){
  int n=sz(a),L=__builtin_ctz(n);
  static vc<complex<long double>>R(2,1);
  static vc<C>rt(2,1);
  for(static int k=2;k<n;k*=2){
    R.resize(n);rt.resize(n);
    auto x=polar(1.0L,acosl(-1)/k);
    fwd(i,k,2*k)rt[i]=R[i]=i&1?R[i/2]*x:R[i/2];
  }
  vi rev(n);
  rep(i,n)rev[i]=(rev[i/2]|(i&1)<<L)/2;
  rep(i,n)if(i<rev[i])swap(a[i],a[rev[i]]);
  for(int k=1;k<n;k*=2)for(int i=0;i<n;i+=2*k)rep(j,k){
    C z=rt[j+k]*a[i+j+k];
    a[i+j+k]=a[i+j]-z;
    a[i+j]+=z;
  }
}

vl multiply(const vi&a,const vi&b){
  if(a.empty()||b.empty())return{};
  vl res(sz(a)+sz(b)-1);
  int n=1;
  while(n<sz(res))n*=2;
  vc<C>in(n),out(n);
  rep(i,sz(a))in[i].real(a[i]);
  rep(i,sz(b))in[i].imag(b[i]);
  fft(in);
  for(C&x:in)x*=x;
  rep(i,n)out[i]=in[-i&(n-1)]-conj(in[i]);
  fft(out);
  rep(i,sz(res))res[i]=llround(imag(out[i])/(4*n));
  return res;
}
```

gauss

```
// gauss(a,p): a has n rows of m coefficients followed by the right side, all in [0,p) for prime p
// returns any solution of the system modulo p (free variables are 0), or empty vector if there is none
ll mpow(ll b,ll e,ll p){
  ll r=1;
  for(;e;e/=2,b=b*b%p)if(e&1)r=r*b%p;
  return r;
}

vl gauss(vc<vl>a,ll p){
  int n=sz(a),m=sz(a[0])-1,r=0;
  vi where(m,-1);
  for(int c=0;c<m&&r<n;c++){
    int s=r;
    while(s<n&&!a[s][c])s++;
    if(s==n)continue;
    swap(a[s],a[r]);
    ll inv=mpow(a[r][c],p-2,p);
    for(ll&v:a[r])v=v*inv%p;
    rep(i,n)if(i!=r&&a[i][c]){
      ll f=a[i][c];
      fwd(j,c,m+1)a[i][j]=(a[i][j]+(p-f)*a[r][j])%p;
    }
    where[c]=r++;
  }
  fwd(i,r,n)if(a[i][m])return{};
  vl x(m);
  rep(c,m)if(where[c]>=0)x[c]=a[where[c]][m];
  return x;
}
```

geometry

```
// pt: point with integer coordinates (absolute value at most 1e9)
// line{a,b}: infinite line through a!=b, seg{a,b}: segment from a to b (a==b allowed)
// poly: vertices of a simple polygon in order (any orientation), counts as a filled region
// intersect(x,y): whether x and y have a common point, x and y are any of pt, line, seg, poly
// inside(g,p): 0 if p is outside polygon g, 1 if on its boundary, 2 if strictly inside
// side(a,b,c): 1 if a->b->c turns left, -1 if right, 0 if collinear
struct pt{
  ll x,y;
  pt operator-(pt o)const{return{x-o.x,y-o.y};}
  bool operator==(pt o)const{return x==o.x&&y==o.y;}
};
struct line{pt a,b;};
struct seg{pt a,b;};
typedef vc<pt> poly;

ll cross(pt a,pt b){return a.x*b.y-a.y*b.x;}
ll dot(pt a,pt b){return a.x*b.x+a.y*b.y;}
int side(pt a,pt b,pt c){ll v=cross(b-a,c-a);return(v>0)-(v<0);}

bool intersect(pt a,pt b){return a==b;}
bool intersect(pt p,line l){return!side(l.a,l.b,p);}
bool intersect(pt p,seg s){return!side(s.a,s.b,p)&&dot(s.a-p,s.b-p)<=0;}
bool intersect(line l,line m){return cross(l.b-l.a,m.b-m.a)||!side(l.a,l.b,m.a);}
bool intersect(line l,seg s){return side(l.a,l.b,s.a)*side(l.a,l.b,s.b)<=0;}
bool intersect(seg s,seg t){
  if(side(s.a,s.b,t.a)*side(s.a,s.b,t.b)<0&&side(t.a,t.b,s.a)*side(t.a,t.b,s.b)<0)return 1;
  return intersect(t.a,s)||intersect(t.b,s)||intersect(s.a,t)||intersect(s.b,t);
}

int inside(const poly&g,pt p){
  bool in=0;
  rep(i,sz(g)){
    pt a=g[i],b=g[(i+1)%sz(g)];
    if(intersect(p,seg{a,b}))return 1;
    if((a.y>p.y)!=(b.y>p.y)&&(side(a,b,p)>0)==(b.y>a.y))in^=1;
  }
  return 2*in;
}

bool intersect(pt p,const poly&g){return inside(g,p);}
bool intersect(line l,const poly&g){
  int lo=1,hi=-1;
  for(pt p:g)lo=min(lo,side(l.a,l.b,p)),hi=max(hi,side(l.a,l.b,p));
  return lo<=0&&hi>=0;
}
bool intersect(seg s,const poly&g){
  if(inside(g,s.a))return 1;
  rep(i,sz(g))if(intersect(s,seg{g[i],g[(i+1)%sz(g)]}))return 1;
  return 0;
}
bool intersect(const poly&g,const poly&h){
  if(inside(g,h[0])||inside(h,g[0]))return 1;
  rep(i,sz(g))rep(j,sz(h))if(intersect(seg{g[i],g[(i+1)%sz(g)]},seg{h[j],h[(j+1)%sz(h)]}))return 1;
  return 0;
}
template<class A,class B>bool intersect(const A&a,const B&b){return intersect(b,a);}
```

hash_string

```
// FastString(s): get(l,r) returns hash of s[l..r-1], equal substrings have equal hashes
typedef unsigned long long ull;
const ull HM=(1ull<<61)-1,HB=chrono::steady_clock::now().time_since_epoch().count()%HM;
ull hmul(ull a,ull b){
  __uint128_t c=(__uint128_t)a*b;
  ull r=(c&HM)+(c>>61);
  return r>=HM?r-HM:r;
}

struct FastString{
  vc<ull>h,p;
  FastString(const string&s):h(sz(s)+1),p(sz(s)+1,1){
    rep(i,sz(s))h[i+1]=(hmul(h[i],HB)+s[i])%HM,p[i+1]=hmul(p[i],HB);
  }
  ull get(int l,int r){
    return(h[r]+HM-hmul(h[l],p[r-l]))%HM;
  }
};
```

li_chao

```
// LCT(n): set of lines on integer x in [0,n)
// add_line({k,m}): adds line y=k*x+m, get(x): minimum of all lines at x (1e18 if there are none)
struct LCT{
  int n;
  vc<pll>t;
  LCT(int n):n(n),t(4*n,{0,(ll)1e18}){}
  ll ev(pll a,ll x){return a.f1*x+a.f2;}
  void add_line(pll w,int v=1,int l=0,int r=0){
    if(!r)r=n;
    int m=(l+r)/2;
    bool L=ev(w,l)<ev(t[v],l),M=ev(w,m)<ev(t[v],m);
    if(M)swap(t[v],w);
    if(r-l>1)L!=M?add_line(w,2*v,l,m):add_line(w,2*v+1,m,r);
  }
  ll get(int x,int v=1,int l=0,int r=0){
    if(!r)r=n;
    int m=(l+r)/2;
    ll y=ev(t[v],x);
    return r-l==1?y:min(y,x<m?get(x,2*v,l,m):get(x,2*v+1,m,r));
  }
};
```

max_flow

```
// MaxFlow(n): graph on nodes 0..n-1
// add(a,b,c): directed edge a->b with capacity c
// maxflow(s,t): returns max flow from s to t, call only once
// after maxflow, d[v]>=0 iff v is on the source side of a min cut
struct MaxFlow{
  struct E{int to;ll c;};
  vc<E>e;vc<vi>g;vi d,it;
  MaxFlow(int n):g(n),d(n),it(n){}
  void add(int a,int b,ll c){
    g[a].pb(sz(e));e.pb({b,c});
    g[b].pb(sz(e));e.pb({a,0});
  }
  ll dfs(int u,int t,ll f){
    if(u==t||!f)return f;
    ll r=0;
    for(int&i=it[u];i<sz(g[u]);i++){
      E&x=e[g[u][i]];
      if(d[x.to]==d[u]+1)if(ll p=dfs(x.to,t,min(f-r,x.c))){
        x.c-=p;e[g[u][i]^1].c+=p;
        if((r+=p)==f)return r;
      }
    }
    return r;
  }
  ll maxflow(int s,int t){
    ll f=0;
    while(1){
      fill(all(d),-1);fill(all(it),0);
      d[s]=0;queue<int>q;q.push(s);
      while(sz(q)){
        int u=q.front();q.pop();
        for(int i:g[u])if(e[i].c&&d[e[i].to]<0)d[e[i].to]=d[u]+1,q.push(e[i].to);
      }
      if(d[t]<0)return f;
      f+=dfs(s,t,LLONG_MAX);
    }
  }
};
```

min_cost_flow

```
// MinCostFlow(n): graph on nodes 0..n-1
// add_edge(a,b,cap,cost): directed edge a->b
// min_cost_flow(s,t,K): sends at most K flow from s to t as cheaply as possible, returns {flow,cost}
// negative costs are allowed if there is no negative cycle
struct MinCostFlow{
  struct E{int to;ll c,w;};
  int n;vc<E>e;vc<vi>g;
  MinCostFlow(int n):n(n),g(n){}
  void add_edge(int a,int b,ll cap,ll cost){
    g[a].pb(sz(e));e.pb({b,cap,cost});
    g[b].pb(sz(e));e.pb({a,0,-cost});
  }
  pll min_cost_flow(int s,int t,ll K=LLONG_MAX){
    ll f=0,c=0;
    vl pi(n,LLONG_MAX);vi inq(n);
    queue<int>q;q.push(s);pi[s]=0;
    while(sz(q)){
      int u=q.front();q.pop();inq[u]=0;
      for(int i:g[u])if(e[i].c&&pi[u]+e[i].w<pi[e[i].to]){
        pi[e[i].to]=pi[u]+e[i].w;
        if(!inq[e[i].to])inq[e[i].to]=1,q.push(e[i].to);
      }
    }
    while(f<K&&pi[t]<LLONG_MAX){
      vl d(n,LLONG_MAX);vi p(n);
      priority_queue<pll,vc<pll>,greater<>>pq;pq.push({0,s});d[s]=0;
      while(sz(pq)){
        auto[du,u]=pq.top();pq.pop();
        if(du>d[u])continue;
        for(int i:g[u]){
          E&x=e[i];
          if(!x.c)continue;
          ll nd=du+x.w+pi[u]-pi[x.to];
          if(nd<d[x.to])d[x.to]=nd,p[x.to]=i,pq.push({nd,x.to});
        }
      }
      if(d[t]==LLONG_MAX)break;
      rep(v,n)if(d[v]<LLONG_MAX)pi[v]+=d[v];
      ll r=K-f;
      for(int v=t;v!=s;v=e[p[v]^1].to)r=min(r,e[p[v]].c);
      for(int v=t;v!=s;v=e[p[v]^1].to)e[p[v]].c-=r,e[p[v]^1].c+=r;
      f+=r;c+=r*pi[t];
    }
    return{f,c};
  }
};
```

modulo_int

```
// M: integer modulo MOD (MOD must be prime for division)
// supports + - * / and assignment versions, pow, == != <, cin, cout and use as key in set, map and unordered_map
const ll MOD=1e9+7;
struct M{
  ll val;
  M(ll x=0):val((x%MOD+MOD)%MOD){}
  M&operator+=(M b){if((val+=b.val)>=MOD)val-=MOD;return*this;}
  M&operator-=(M b){if((val-=b.val)<0)val+=MOD;return*this;}
  M&operator*=(M b){val=val*b.val%MOD;return*this;}
  M&operator/=(M b){return*this*=b.pow(MOD-2);}
  friend M operator+(M a,M b){return a+=b;}
  friend M operator-(M a,M b){return a-=b;}
  friend M operator*(M a,M b){return a*=b;}
  friend M operator/(M a,M b){return a/=b;}
  friend bool operator==(M a,M b){return a.val==b.val;}
  friend bool operator!=(M a,M b){return a.val!=b.val;}
  bool operator<(M b)const{return val<b.val;}
  M pow(ll e)const{M r=1,a=*this;for(;e;e/=2,a*=a)if(e&1)r*=a;return r;}
  friend ostream&operator<<(ostream&s,M m){return s<<m.val;}
  friend istream&operator>>(istream&s,M&m){ll x;s>>x;m=x;return s;}
};
template<>struct std::hash<M>{size_t operator()(M x)const{return x.val;}};
```

ntt

```
// ntt_multiply(a,b): coefficients of the product of polynomials a and b modulo NM, coefficients must be in [0,NM)
// works for any prime NM=c*2^k+1 with primitive root NG while the result has at most 2^k coefficients
const ll NM=998244353,NG=3;
ll npow(ll b,ll e){
  ll r=1;
  for(;e;e/=2,b=b*b%NM)if(e&1)r=r*b%NM;
  return r;
}

void ntt(vl&a,bool inv){
  int n=sz(a);
  for(int i=1,j=0;i<n;i++){
    int bit=n>>1;
    for(;j&bit;bit>>=1)j^=bit;
    j^=bit;
    if(i<j)swap(a[i],a[j]);
  }
  for(int k=1;k<n;k*=2){
    ll w=npow(NG,(NM-1)/(2*k));
    if(inv)w=npow(w,NM-2);
    for(int i=0;i<n;i+=2*k){
      ll z=1;
      rep(j,k){
        ll u=a[i+j],v=a[i+j+k]*z%NM;
        a[i+j]=u+v<NM?u+v:u+v-NM;
        a[i+j+k]=u-v>=0?u-v:u-v+NM;
        z=z*w%NM;
      }
    }
  }
  if(inv){
    ll ni=npow(n,NM-2);
    for(ll&x:a)x=x*ni%NM;
  }
}

vl ntt_multiply(vl a,vl b){
  if(a.empty()||b.empty())return{};
  int s=sz(a)+sz(b)-1,n=1;
  while(n<s)n*=2;
  a.resize(n);b.resize(n);
  ntt(a,0);ntt(b,0);
  rep(i,n)a[i]=a[i]*b[i]%NM;
  ntt(a,1);
  a.resize(s);
  return a;
}
```

setup

```
/*

setxkbmap us -option caps:escape

~/.vimrc

set number relativenumber tabstop=2 shiftwidth=2 autoindent
syntax enable
filetype plugin indent on

Makefile:

%: %.cpp
  g++ -std=c++20 -O2 -Wall -DLOC $< -o $@

 */
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
#define fwd(i,a,b) for(int i=(a); i<(b);i++)
#define rep(i,n) fwd(i,0,n)
#define all(x) (x).begin(), (x).end()
#define sz(x) ((int)(x).size())
#define f1 first
#define f2 second
#define pb push_back
#define vc vector
typedef vc<int> vi;
typedef vc<ll> vl;
typedef pair<int,int> pii;
typedef pair<ll,ll> pll;

#ifdef LOC
template<class A,class B>auto&operator<<(ostream&o,pair<A,B>p){return o<<"("<<p.f1<<", "<<p.f2<<")";}
template<class T>auto&operator<<(ostream&o,const vc<T>&v){o<<"{";for(int i=0;auto&x:v)o<<(i++?", ":"")<<x;return o<<"}";}
#define debug(x) cerr<<"[" #x "]: "<<x<<"\n";
#else
#define debug(x)
#endif

void solve(){
}

int main(){
  cin.tie(0);cout.tie(0);ios::sync_with_stdio(false);
  solve();return 0;
}
```

suffix_array

```
// suffix_array(s): starting indices of all suffixes of s in sorted order, s must not contain '\0' or negative chars
vi suffix_array(string s){
  s.pb(0);
  int n=sz(s);
  vi p(n),c(all(s)),t(n),cnt(max(256,n));
  iota(all(p),0);
  sort(all(p),[&](int a,int b){return c[a]<c[b];});
  for(int k=1;;k*=2){
    rep(i,n)t[i]=(p[i]-k+n)%n;
    fill(all(cnt),0);
    for(int x:c)cnt[x]++;
    partial_sum(all(cnt),cnt.begin());
    for(int i=n;i--;)p[--cnt[c[t[i]]]]=t[i];
    t[p[0]]=0;
    fwd(i,1,n)t[p[i]]=t[p[i-1]]+(pii(c[p[i]],c[(p[i]+k)%n])!=pii(c[p[i-1]],c[(p[i-1]+k)%n]));
    c.swap(t);
    if(c[p[n-1]]==n-1)break;
  }
  p.erase(p.begin());
  return p;
}

// lcp(s,p): with p=suffix_array(s), l[i] is the longest common prefix of suffixes p[i] and p[i+1]
vi lcp(const string&s,const vi&p){
  int n=sz(s),k=0;
  vi r(n),l(n-1);
  rep(i,n)r[p[i]]=i;
  rep(i,n)if(r[i]<n-1){
    int j=p[r[i]+1];
    while(max(i,j)+k<n&&s[i+k]==s[j+k])k++;
    l[r[i]]=k;
    k-=!!k;
  }else k=0;
  return l;
}
```

tarjan_bridge

```
// define maxN, add undirected edges to nodes[], then call tarjan_bridge(root)
// bridges then holds every bridge {u,v} of the component of root, parallel edges are handled
int timer,tin[maxN],low[maxN];
vi nodes[maxN];
vc<pii>bridges;

void tarjan_bridge(int u=0,int p=-1){
  tin[u]=low[u]=++timer;
  bool s=0;
  for(int v:nodes[u]){
    if(v==p&&!s)s=1;
    else if(tin[v])low[u]=min(low[u],tin[v]);
    else{
      tarjan_bridge(v,u);
      low[u]=min(low[u],low[v]);
      if(low[v]>tin[u])bridges.pb({u,v});
    }
  }
}
```

two_sat

```
// TwoSat(n): n boolean variables 0..n-1, literal x means x is true and ~x means x is false
// either(a,b): adds clause a or b
// solve(): returns whether all clauses can be satisfied, val[i] is then a valid value of variable i
struct TwoSat{
  int n,t=0,c=0;vc<vi>g;vi tin,low,comp,st,val;
  TwoSat(int n):n(n),g(2*n){}
  int id(int x){return x<0?2*~x+1:2*x;}
  void either(int a,int b){
    g[id(a)^1].pb(id(b));
    g[id(b)^1].pb(id(a));
  }
  void dfs(int u){
    tin[u]=low[u]=++t;st.pb(u);
    for(int v:g[u]){
      if(!tin[v])dfs(v),low[u]=min(low[u],low[v]);
      else if(comp[v]<0)low[u]=min(low[u],tin[v]);
    }
    if(low[u]==tin[u]){
      int v;
      do v=st.back(),st.pop_back(),comp[v]=c;while(v!=u);
      c++;
    }
  }
  bool solve(){
    tin.assign(2*n,0);low=tin;comp.assign(2*n,-1);val.resize(n);
    rep(i,2*n)if(!tin[i])dfs(i);
    rep(i,n){
      if(comp[2*i]==comp[2*i+1])return 0;
      val[i]=comp[2*i]<comp[2*i+1];
    }
    return 1;
  }
};
```

z_algorithm

```
// z_function(s): z[i] is the length of the longest common prefix of s and s[i..], z[0]=0
vi z_function(const string&s){
  int n=sz(s),l=0,r=0;
  vi z(n);
  fwd(i,1,n){
    if(i<r)z[i]=min(r-i,z[i-l]);
    while(i+z[i]<n&&s[z[i]]==s[i+z[i]])z[i]++;
    if(i+z[i]>r)l=i,r=i+z[i];
  }
  return z;
}
```